Feature Extraction

In order to feed text to a model we need to transform it to a numerical features.

One Hot Encoding using scikit -learn

```
%pip install scikit-learn
```

We encode our corpus as a one-hot numeric array using scikit-learn's OneHotEncoder. We will demostrate:

- One Hot Encoding: In one-hot encoding, each word w in corpus vocabulary is given a unique integer id wid that is between 1 and |V|, where V is the set of corpus vocab. Each word is then represented by a V-dimensional binary vector of 0s and 1s.
- Label Encoding: In Label Encoding, each word w in our corpus is converted into a numeric value between 0 and n-1 (where n refers to number of unique words in our corpus).

```
D1 = 'dog bites man'
D2 = 'man bites dog'
D3 = 'dog eats meat'
D4 = 'man eats food'
```

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

data = [D1.split(), D2.split(), D3.split(), D4.split()]
print(data)
values = data[0]+data[1]+data[2]+data[3]
print("The data: ",values)
```

```
[['dog', 'bites', 'man'], ['man', 'bites', 'dog'], ['dog', 'eats', 'meat'], ['man', 'eats', 'food']]
The data:  ['dog', 'bites', 'man', 'man', 'bites', 'dog', 'dog', 'eats', 'meat', 'man', 'eats', 'food']
```

```
#Label Encoding
label_encoder = LabelEncoder()
integer_encoded = label_encoder.fit_transform(values)

print("Label Encoded:",integer_encoded)
```

```
Label Encoded: [1 0 4 4 0 1 1 2 5 4 2 3]
```

```
#One-Hot Encoding
import numpy as np
values_2d = np.array(data).reshape(-1,1)
onehot_encoder = OneHotEncoder()
onehot_encoded = onehot_encoder.fit_transform(values_2d).toarray()

print("Onehot Encoded Matrix:\n",onehot_encoded)
```

```
Onehot Encoded Matrix:
 [[0. 1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1. 0.]
 [1. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 1. 0.]
 [0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0.]]
```

Bag of words

Count the occurrences of words in the corpus.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
texts = [ 'the red dog', 'cat eats dog', 'dog eats food',
         'red cat eats', 'the hot dog']

vectorizer = CountVectorizer()
```

```
vectorizer.fit(texts)
x = vectorizer.transform(texts)

columns = vectorizer.get_feature_names_out()
pd.DataFrame(x.todense(), columns=columns, index=texts)
```

	cat	dog	eats	food	hot	red	the
the red dog	0	1	0	0	0	1	1
cat eats dog	1	1	1	0	0	0	0
dog eats food	0	1	1	1	0	0	0
red cat eats	1	0	1	0	0	1	0
the hot dog	0	1	0	0	1	0	1

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
texts = [ 'the red dog', 'cat eats dog', 'dog eats food',
         'red cat eats', 'the hot dog']

vectorizer = CountVectorizer()
x = vectorizer.fit_transform(texts)
columns = vectorizer.get_feature_names_out()
pd.DataFrame(x.todense(), columns=columns, index=texts)
```

	cat	dog	eats	food	hot	red	the
the red dog	0	1	0	0	0	1	1
cat eats dog	1	1	1	0	0	0	0
dog eats food	0	1	1	1	0	0	0
red cat eats	1	0	1	0	0	1	0
the hot dog	0	1	0	0	1	0	1

```
#Get the representation using this vocabulary, for a new text
temp = vectorizer.transform(["dog and dog are friends"])
print("Bow representation for 'dog and dog are friends':", temp.toarray())
```

```
Bow representation for 'dog and dog are friends': [[0 2 0 0 0 0 0]]
```

```
#BoW with binary vectors
count_vect = CountVectorizer(binary=True)
count_vect.fit(texts)
temp = count_vect.transform(["dog and dog are friends"])
print("Bow representation for 'dog and dog are friends':", temp.toarray())
```

```
Bow representation for 'dog and dog are friends': [[0 1 0 0 0 0 0]]
```

Stop-words

Stop-words are words that are not significant to the topic in hand, for example `[am, is, are, in, at, ...]` can be considered stop-words in many applications as they don't add meaning.

In some other domains and problems you may have different kind of stop-words, for example if you are processing some chatbot data you may find `[can you please, would you please, can I, may I, ...]` such examples don't add meaning so stop-words can also be domain specific, and `TFIDF` can help you find these.

```
texts = [ 'the red dog', 'cat eats dog', 'dog eats food',
         'red cat eats', 'the hot dog']

vectorizer = CountVectorizer(stop_words='english')
vectorizer.fit(texts)
x = vectorizer.transform(texts)
columns = vectorizer.get_feature_names_out()
pd.DataFrame(x.todense(), columns=columns, index=texts)
```

	cat	dog	eats	food	hot	red
the red dog	0	1	0	0	0	1
cat eats dog	1	1	1	0	0	0
dog eats food	0	1	1	1	0	0
red cat eats	1	0	1	0	0	1
the hot dog	0	1	0	0	1	0

> Note that the word `the` was removed here

N-Grams

N-Grams is a way we can use to count for the context in the text, the bigger n-gram range the bigger context you can capture but also more features to generate, so be careful not to break your memory.

```
texts = [ 'the red dog', 'cat eats dog', 'dog eats food',
         'red cat eats', 'the hot dog']

vectorizer = CountVectorizer(stop_words='english', ngram_range=(1, 3))
vectorizer.fit(texts)
x = vectorizer.transform(texts)
columns = vectorizer.get_feature_names_out()
pd.DataFrame(x.todense(), columns=columns, index=texts)
```

	cat	cat eats	cat eats dog	dog	dog eats	dog eats food	eats	eats dog	eats food	food	hot	hot dog	red	red cat	red cat eats	red dog
the red dog	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	1
cat eats dog	1	1	1	1	0	0	1	1	0	0	0	0	0	0	0	0
dog eats food	0	0	0	1	1	1	1	0	1	1	0	0	0	0	0	0

TFIDF

Instead of just counting the frequency of each word, each word here is weighted using TF-IDF

$$W_{x,y} = tf_{x,y} \times log(\frac{N}{df_x})$$

```
from sklearn.feature_extraction.text import TfidfVectorizer
texts = [ 'the red dog', 'cat eats dog', 'dog eats food',
         'red cat eats', 'the hot dog']

vectorizer = TfidfVectorizer()
x = vectorizer.fit_transform(texts)
columns = vectorizer.get_feature_names_out()
pd.DataFrame(x.todense(), columns=columns, index=texts)
```

	cat	dog	eats	food	hot	red	the
the red dog	0.000000	0.442740	0.000000	0.000000	0.000000	0.634027	0.634027
cat eats dog	0.677803	0.473309	0.562638	0.000000	0.000000	0.000000	0.000000
dog eats food	0.000000	0.423954	0.503968	0.752515	0.000000	0.000000	0.000000
red cat eats	0.609818	0.000000	0.506204	0.000000	0.000000	0.609818	0.000000
the hot dog	0.000000	0.401565	0.000000	0.000000	0.712775	0.000000	0.575063

```
vectorizer.idf_
```

```
array([1.69314718, 1.18232156, 1.40546511, 2.09861229, 2.09861229,
       1.69314718, 1.69314718])
```

```
vectorizer.vocabulary_
```

```
{'red': 10,
 'dog': 2,
 'red dog': 12,
 'cat': 0,
 'eats': 4,
 'cat eats': 1,
 'eats dog': 5,
 'food': 7,
 'dog eats': 3,
 'eats food': 6,
 'red cat': 11,
 'hot': 8,
 'hot dog': 9}
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
texts = [ 'the red dog', 'cat eats dog', 'dog eats food',
         'red cat eats', 'the hot dog']

vectorizer = TfidfVectorizer(stop_words='english', ngram_range=(1, 2))
vectorizer.fit(texts)
x = vectorizer.transform(texts)
columns = vectorizer.get_feature_names_out()
pd.DataFrame(x.todense(), columns=columns, index=texts)
```

	cat	cat eats	dog	dog eats	eats	eats dog	eats food	food	hot	hot dog	red	red cat	red dog
the red dog	0.000000	0.000000	0.401565	0.000000	0.000000	0.00000	0.000000	0.000000	0.0000	0.0000	0.575063	0.000000	0.712775
cat eats dog	0.460631	0.460631	0.321658	0.000000	0.382365	0.57094	0.000000	0.000000	0.0000	0.0000	0.000000	0.000000	0.000000

```
[(idf,word) for idf,word in zip(vectorizer.get_feature_names_out(),vectorizer.idf_)]
```

```
[('cat', 1.6931471805599454),
 ('cat eats', 1.6931471805599454),
 ('dog', 1.1823215567939547),
 ('dog eats', 2.09861228866811),
 ('eats', 1.4054651081081644),
 ('eats dog', 2.09861228866811),
 ('eats food', 2.09861228866811),
 ('food', 2.09861228866811),
 ('hot', 2.09861228866811),
 ('hot dog', 2.09861228866811),
 ('red', 1.6931471805599454),
 ('red cat', 2.09861228866811),
 ('red dog', 2.09861228866811)]
```

```
temp = vectorizer.transform(["dog and cat are friends"])
print("Tfidf representation for 'dog and cat are friends':\n", temp.toarray())
```

```
Tfidf representation for 'dog and cat are friends':
 [[0.8198869  0.         0.57252551 0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.        ]]
```